

Module: Problem 9

This module provides a utility function for checking whether two collections contain the same elements (i.e., are permutations of each other). When they are permutations, the function also reports the most frequent element, its occurrence count, and its type.

Functions

`is_list_permutation(L1, L2)`

Description

Determines whether `L1` and `L2` are permutations of one another.

If the multisets of elements differ, the function returns `False`.

If they are permutations, it returns a three-item tuple containing:

1. the element that occurs most often,
2. the number of its occurrences, and
3. the Python type of that element (`type(value)`).

If both inputs are empty (i.e., the counter is empty), the function returns `(None, None, None)`.

Parameters

- `L1` (*iterable of `int` or `str`*): The first collection to compare.
- `L2` (*iterable of `int` or `str`*): The second collection to compare.

Returns

- `False` if `L1` and `L2` are **not** permutations of each other.
- `tuple(value, count, type)` where `value` is the most frequent element, `count` is its frequency, and `type` is `type(value)`, when they are permutations.
- `(None, None, None)` when both inputs are empty.

Examples

```
# Both inputs are the same string; Counter treats the string as an iterable of characters
result = is_list_permutation('test', 'test')
# → ('t', 2, <class 'str'>)

# Both inputs are empty strings → empty Counter
result = is_list_permutation('', '')
# → (None, None, None)

# Non-permutations
result = is_list_permutation([1, 2, 3], [3, 2, 4])
# → False
```

Edge Cases / Notes - The function uses `collections.Counter`, which expects each argument to be **iterable**.

Supplying non-iterable objects (e.g., integers) raises a `TypeError`: `python is_list_permutation(0, 0) # TypeError: 'int' object is not iterable is_list_permutation(1, 1) # TypeError: 'int' object is not iterable is_list_permutation(-1, -1) # TypeError: 'int' object is not iterable` - When strings are passed, the function counts **characters**, not whole strings. This may differ from the intended use with lists

of strings. - An empty input (e.g., `''` or `[]`) results in an empty `Counter`, and the function returns `(None, None, None)`.